

# 44. Směrování a zabezpečení přenosů v počítačových sítích (Link-State, Distance-Vector, šifrování, autentizace a integrity dat)

## 44. Směrování a zabezpečení přenosů

SZZ okruh 44 (FIT BUT). Jak se paket dostane sítí a jak je chráněn.

### Shrnutí

#### Směrování

- **Směrování (L3, routery)** mezi sítěmi × **přepínání (L2, switche)** v rámci sítě; směrovací tabulka.
- Třídní × **beztrídní** (CIDR, agregace, **Longest Prefix Match**).
- **Link-State** (OSPF, IS-IS) — každý router zná **celou topologii** (flooding) □ **Dijkstra** ([[okruhy/25-teorie-grafu]]).
- **Distance-Vector** (RIP, EIGRP) — router komunikuje **jen se sousedy**, iterativně; problém *count-to-infinity*.

#### Šifrování (důvěrnost)

- **Symetrická** — jeden tajný klíč (AES, ChaCha20); rychlá; klíč □ 128–256 b.
- **Asymetrická** — pár veřejný/soukromý (RSA, ECC); šifruje se veřejným, dešifruje soukromým; pomalejší, delší klíče.
- TLS: výměna klíčů asymetricky, pak přechod na symetrickou (výkon).

#### Integrita, podpis, certifikáty

- **Integrita** — kryptografický **hash** (charakteristika zprávy; odolnost vůči kolizím, nevratnost); SHA-3, BLAKE2.
- **Digitální podpis** (neodmítnutelnost + autenticita + integrita): hash zprávy **šifrovaný privátním** klíčem; ověření veřejným. **Hash** ≠ **šifra** ≠ **podpis**.
- **Certifikát** — váže veřejný klíč k identitě; podepsán **certifikační autoritou (CA)**; **řetězec důvěry** ke kořenové CA.
- **Replay attack** — ochrana časovými razítky / sekvenčními čísly.

Transportní/síťová vrstva viz [[okruhy/43-tcp-ip-komunikace]]; Dijkstra viz [[okruhy/25-teorie-grafu]]; TLS/JWT a autentizace viz [[okruhy/37-webova-rozhrani-autentizace]].

#### Související syntéza

- [[synthesis/kryptografie-autentizace|Kryptografie × autentizace]] — syntéza

## Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

**Směrování** □ [[#Směrování]] - Co dělá router, co je ve směrovací tabulce? □ Směruje pakety mezi sítěmi (L3); tabulka = síťové adresy + výstup/další skok. - Distance-Vector vs. Link-State? □ DV komunikuje jen se sousedy (RIP), iterativně; LS zná celou topologii (OSPF) + Dijkstra.

**Šifrování** □ [[#Šifrování (důvěrnost)]] - Symetrické vs. asymetrické? □ Sym = 1 klíč, rychlé; asym = pár klíčů (veřejný/soukromý), pomalejší. TLS kvůli výkonu přechází na symetrické.

**Integrita a podpis** □ [[#Integrita, podpis, certifikáty]] - Hash vs. šifra vs. podpis? □ Hash = charakteristika (integrita, nevratný); šifra = důvěrnost (obousměrné); podpis = hash šifrovaný privátním klíčem (autenticita). - Digitální podpis krok za krokem? □ hash zprávy □ zašifrovat privátním klíčem □ příjemce dešifruje veřejným a porovná s vlastním hashem. - Certifikáty a CA? □ CA podepisuje veřejný klíč □ váže ho k identitě; řetězec důvěry ke kořenové CA. - Replay attack? □ Opakované zaslání zachycené zprávy; ochrana časovými razítky / sekvenčními čísly.

## Plné znění (ke studiu)

Směrování (routing) je **určování cest paketů** mezi různými počítačovými sítěmi (které jsou ale na jednom internetu). Směrování zajišťují **směrovače** (routery - pomocí **IP adres** na **síťové vrstvě - L3**). Úkolem směrování je doručit IP paket adresátovi (který je v jiné síti), pokud možno co **nejefektivnější** cestou (tj. co nejrychleji).

- V rámci **jedné** sítě se **nesměruje, ale přepíná (switching)**. Přepínání provádí přepínač (**switch**) a děje se na **linkové vrstvě** (L2). Přepínají se **rámce**.
- Pojmy:
  - PDU (protocol data unit) na **linkové vrstvě** je **rámec** (frame),
  - PDU na **síťové vrstvě** je **paket** (packet),
  - PDU na transportní vrstvě je:
    - \* **tok (stream)**, případně **segment** pro TCP (viz SOCK\_STREAM),
    - \* **datagram** pro UDP (viz SOCK\_DGRAM),

Pojem **paket** je ale často také považován za data přijatá přes **TCP** a pojem **datagram** jako data přijatá přes **UDP** a případně jinými způsoby s možnou ztrátou.

## Třídní směrování (Classful routing)

[[media/szz-44/media/image1.png]]

Položky ve **směrovací tabulce** (routing table) tvoří **pouze síťové adresy** o délce **8, 16 a 24** bitů, viz obrázek vlevo. Délka vychází z **masek sítí** jednotlivých tříd. Třidu lze poznat dle nejvyšších bitů (prefixu), viz obrázek vpravo. V routing table je  $2^7 + 2^{14} + 2^{21} = 2\,113\,664$  adres (ne všechny jsou použitelné, např. broadcastové), ale i tak jich je mnoho. [[media/szz-44/media/image10.png]]

## Beztrídní směrování (Classless routing)

Položky ve směrovací tabulce tvoří **síťové adresy** a **maska sítě**. Pro každou adresu je na základě **masky sítě** nejdříve určena **síťová adresa**, až pak je provedeno směrování **158.193.138.40 & 255.255.255.224 = 158.193.138.32**.

Pro to, aby se zmenšily směrovací tabulky, **agregují** směrovače adresy sítí, pokud se pro jednu cestu **objeví více adres s rozdílnými LSB** a eliminuje se tak potřeba je rozlišovat. 

Na obrázku mohl směrovač B agregovat všechny 4 IP adresy do jedné a snížit tak masku sítě na 22, protože IP adresy obsahují na bitu 9 a 8 všechny kombinace (**00, 01, 10 i 11**). To ale znamená, že když na směrovač přichází teď adresa s maskou 24, není v tabulce. Výběr cesty, kam bude takový paket poslán, pak probíhá na základě vyhledání **nejdelší shody prefixu** - adresy sítě (**Longest Prefix Match**). 



## Link-State routing

Jedná se např. o protokoly **Open Shortest Path First (OSPF)** a **Intermediate System to Intermediate System (IS-IS)**. Link-state směrování je založeno na tom, že každý **směrovač** (router) zná **nejlepší cestu** (cestu s nejnižší cenou dle nějakého hodnocení - rychlost přenosu mezi routery a počet routerů po cestě) do **všech sítí**. Tzn. že každý směrovač **zná celou topologii sítě**. Dosáhneme toho:

- **Flooding**: Každý směrovač **broadcastuje** (posílá všem) Link-State paket obsahující **ceny** cest k jeho **sousedním** směrovačům. Každý směrovač si **ukládá** informace (ceny cest) o směrování od ostatních směrovačů. Pokud směrovači přijde Link-State paket, který **už jednou dostal**, již jej dále **nepřeposílá** (už jej jednou přeposlal). **Problémem** může být **ztráta** Link-State paketů, některý směrovač tak nemusí mít kompletní přehled o síti. Řeší se pomocí **ACK** a případného přeposlání.
- Flooding je nutné **provádět** při **přidání** nového směrovače, při **odebrání** a případně také **periodicky**, pokud se ceny cest mohou měnit.

Jakmile zná každý směrovač celou topologii sítě, má sestavený **vážený** (ohodnocený) **graf**. Nyní musí každý směrovač vypočítat **nejlepší cestu do všech sítí** (jiných směrovačů), používá se na to **Dijkstrův algoritmus** pro hledání nejkratší cesty/cest v ohodnoceném grafu, viz okruh 25. Následně každý router ví, kam má paket směřovat na základě jeho cílové destinace. (Implementovat pak lze tak, že k jednotlivým výstupům si přiřadí všechny IP adresy, které na ten výstup vedou, a provádí porovnávání - pomocí asociativní paměti (adresovatelné obsahem). Pokud navíc jsou si IP adresy podobné, může při porovnávání ignorovat některé bity). Link-State Routing Algorithm - IP Network Control Plane | Computer Networks Ep 5.2.1 | Kurose & Ross

## Distance-Vector routing

Distance-Vector Routing Algorithm - IP Network Layer | Computer Networks Ep. 5.2.2 | Kurose & Ross

Jedná se o protokoly **Enhanced Interior Gateway Routing Protocol (EIGRP)** a **Routing Information Protocol (RIP)**. Každý uzel (směrovač) komunikuje **pouze se svými sousedy** (směrovači, se kterými má přímé spojení). Směrovač tak na začátku algoritmu zná cestu pouze k sousedním směrovačům (do vzdálenosti 1) a zapíše ceny těchto cest do své směrovací tabulky, do zbytku sítě nevidí (na příkladech se to vyjadřuje nekonečným ohodnocením, v realitě ani neví, že tam něco existuje). Algoritmus je **iterativní**.

- **1. iterace**: směrovač (a všechny ostatní také) dostane směrovací tabulky od **sousedních** směrovačů, vidí tak už do **vzdálenosti 2**. Na základě získaných informací **aktualizují svou směrovací tabulku** (přidá **nově zviditelněné** směrovače a pokud se dokáže dostat přes jiný směrovač **rychleji** k uzlu, do kterého již cestu zná, aktualizuje i tuto cestu, stejně tak při zdražení cesty).
- **2. iterace**: směrovač opět dostane směrovací tabulky od svých **sousedních** směrovačů, ty ale **mohou být jiné**, než při 1. iteraci (přibýly nově viditelné směrovače, zlevnili se některé cesty,

zdražily se některé cesty). Směrovač opět aktualizuje svojí směrovací tabulku, už vidí do **vzdálenosti 3**.

- ...
- **n. iterace:** směrovač dostane směrovací tabulky od svých **sousedů**, už ale **vidí do celé sítě** a nově získané informace **nezpůsobí žádnou změnu** v jeho tabulce. Dále už nepřeposílá svojí tabulku, protože se nezměnila. Takhle postupně přestanou odesílat své tabulky všechny směrovače, protože každý už má **ideální směrovací** tabulku.

Z výše uvedeného postupu plyne, že bude třeba **minimálně tolik iterací**, jak je dlouhá **nejdelší nejkratší cesta** (nejkratší znamená, že mezi dvěma uzly nejde již najít nejkratší cestu; nejdelší znamená, že je to pak nejdelší cesta z těchto). Reakce na změny:

- **snížení ceny cesty:** opět si směrovače iterativně vyměňují směrovací tabulky, dokud se směrování neustálí. Probíhá rychle - dobrá zpráva se šíří rychle.
- **zvýšení ceny cesty:** iterativní šíření. Může ale dojít k zajímavému jevu, kdy dva sousední směrovače využívají cesty **přes sebe navzájem** a ty se tak v každé iteraci **zvětšují o 1**, dokud cena nestoupne natolik, že je použita jiná cesta. Např. na obrázku si bude **y** a **z** vyměňovat tabulky, dokud cena **nepřekročí 50**, pak teprve použijí cestu přes **x**. [[media/szz-44/media/image2.png]]

Distance-Vector je **distribuovaný algoritmus**, je problematický, pokud nějaký router **lže** nebo **špatně počítá cenu** (hlásí cenu menší, než doopravdy je). Pak přes něj může být směrován provoz, který ale bude **pomalý** (a takový podvodný router může sledovat všechny provoz, který je přes něj kvůli tomu přenášen).

## Srovnání Link-State a Distance-Vector

[[media/szz-44/media/image3.png]]

## Zabezpečení na síti

[[media/szz-44/media/image6.png]]

Zabezpečením sítě chceme zajistit, aby při komunikaci dvou systémů nedocházelo k:

- **falešnému vydávání se** za jeden ze systémů (**autentizace**),
- **odposlech** jejich komunikace (**důvěrnost**),
- **zásah do komunikace** pozměněním zasílaných zpráv (**integrita dat**),
- **opakované zaslání** zachycené zprávy, která již byla předtím doručena (**replay attack**).

## Autentizace (Authentication)

Autentizace zajišťuje, že uživatel **je tím, za koho se vydává**. Autentizace obvykle probíhá na základě poskytnutí nějaké **tajné informace**. Tuto informaci může znát nebo k ní mít přístup pouze uživatel, který má být autentizován. Jde například o **uživatelské jméno a heslo**, **biometrické údaje**, **bezpečnostní žeton** atd. Autentizaci lze také zajistit **digitálním podpisem**.

## Důvěrnost (Confidentiality)

Důvěrnost zajišťuje, že obsah zpráv **nemůže číst neoprávněná osoba**. To lze zajistit buď odepřením přístupu ke zprávě, což není ale na internetu obecně možné, nebo **šifrováním** (utajením) zprávy, což sice útočníkovi umožní číst zasílaná data, ale nemožní mu získat informaci, kterou obsahují. Data

pro něj **bez dešifrování budou bezcenná**. Nauka o šifrování se nazývá **kryptografie**, v informatice hovoříme především o symetrické a asymetrické kryptografii.

### Symetrická kryptografie

**Symetrická** kryptografie používá **jeden tajný klíč**, kterým odesílatel **šifruje zprávu** a příjemce ji **tím samým klíčem dešifruje**. Nejpoužívanější algoritmy pro symetrické šifrování jsou v současné době například **AES, ChaCha20** či **IDEA**.

Známejší algoritmy jako DES, 3DES a Blowfish nejsou dnes již považované za bezpečné, byla u nich objevena bezpečnostní rizika nebo způsob, jak lze šifru překonat hrubou silou v dostatečně krátkém čase. Síla šifry však závisí především na kvalitě klíče a ta, vzhledem k tomu, že většina **symetrických klíčů** je generována jako **pseudonáhodná posloupnost bitů**, je dána jeho délkou. Za dostatečně bezpečné jsou dnes považovány klíče o délce aspoň **128 bitů**, u kterých není u běžných počítačů reálné jejich prolomení hrubou silou v přijatelně krátkém časovém úseku. Pro zajištění bezpečnosti i do budoucna je ale vhodné používat klíče o délce **256 bitů**.

### Asymetrická kryptografie

U **asymetrické kryptografie** se používá **dvojice klíčů, soukromý a veřejný**. Pro tyto klíče platí, že jsou **matematicky svázány**. Odesílatel **šifruje zprávu veřejným klíčem** příjemce, který může znát **kdokoliv** (buď je veřejně dostupný, nebo jej odesílatel od příjemce získá na počátku komunikace, která není ale šifrovaná, což ovšem nevadí, protože veřejný klíč můžou znát všichni – proto je *veřejný*). Pro dešifrování zprávy je potřeba **soukromý klíč** příjemce (ten zná jen a **pouze příjemce**). Takto je zajištěno šifrování v jednom směru komunikace, pro opačný směr se postupuje obdobně. Obvykle si tedy na počátku komunikace respondenti vymění veřejné klíče a až poté probíhá komunikace šifrovaně. Pro asymetrickou kryptografii se používají například algoritmy **RSA** a **ElGamal**, které jsou založené na složitosti výpočtu **diskrétního logaritmu**, či algoritmus **ECC**, který pracuje na bázi **eliptických křivek**.

Stejně jako u symetrické kryptografie závisí **bezpečnost** šifry především na **délce klíče**. Soukromý klíč lze ale na základě jeho **matematické spojitosti** s veřejným klíčem rekonstruovat rychleji než při rekonstrukci hrubou silou. Proto je nutné používat delší klíče než u symetrické kryptografie. Konkrétně pro algoritmus RSA je bezpečnost klíče o délce 1024 bitů ekvivalentní s bezpečností symetrického klíče o délce 80 bitů, 2048 bitový RSA klíč odpovídá symetrickému klíči o délce 112 bitů a pro zajištění bezpečnosti odpovídající symetrickému klíči o délce 256 bitů je třeba použít RSA klíč o délce 15360 bitů, což už je prakticky nepoužitelná délka. Algoritmus ECC je v tomto směru lepší a pro zajištění bezpečnosti na úrovni 256 bitového symetrického klíče stačí pouze 521 bitový ECC klíč

### Integrita dat (Data Integrity)

Integritou dat ve smyslu zabezpečení komunikace je myšleno zajištění toho, že data po cestě od odesílatele k příjemci nikdo nezmění. Při komunikaci na internetu je toto řešeno vygenerováním **charakteristiky zprávy** (message digest) o fixní délce, připojením této charakteristiky ke zprávě a jejím odesláním. Příjemce nejdříve oddělí charakteristiku zprávy a získá tak původní zprávu, stejným algoritmem jako odesílatel vygeneruje její charakteristiku a porovná ji s obdrženou charakteristikou. Pokud jsou stejné, nikdo po cestě zprávu nezměnil.

Aby výše uvedený princip fungoval, je nutné pro generování charakteristiky použít **kryptografickou hašovací funkci**. Tato funkce zajišťuje, že je velmi náročné vygenerovat zprávu, která by **měla požadovanou charakteristiku**, stejně tak náročné je **najít dvě rozdílné zprávy se stejnou charakteristikou** a na základě charakteristiky není možné **zprávu zrekonstruovat**. Dále taková funkce musí při generování charakteristiky zprávy **zohlednit všechny její bity** a i při změně jediného bitu musí být nově vygenerovaná charakteristika od té původní natolik

**odlišná**, aby se toho nedalo zneužít. Nejpoužívanější bezpečné kryptografické funkce (hashovací funkce) jsou například **MD6, SHA-3** či **BLAKE2**.

I po splnění všech zmíněných předpokladů není stále zajištěno, že zprávu po cestě nikdo nezmění, a to z jednoho prostého důvodu, útočník může změnit zprávu a současně i její charakteristiku. Aby k tomu nedošlo, je nutné použít šifrování, není však nutné šifrovat celou zprávu, stačí **zašifrovat charakteristiku**.

## Neodmítnutelnost (Nonrepudiation)

Neodmítnutelnost zajišťuje, že uživatel nemůže popřít provedení dané akce a poskytuje mu ujištění, že jeho zpráva s požadavkem na provedení této akce byla doručena. K zajištění neodmítnutelnosti se obvykle používá **digitální podpis**. Nejznámějším algoritmem, který implementuje digitální podpis, je **DSA**.

Digitální podpis je založen na **asymetrické kryptografii**, ale na rozdíl od zajištění důvěrnosti se zde pro **šifrování používá privátní klíč**. Privátním klíčem se nešifruje zpráva, pouze její charakteristika, která se získá stejným způsobem jako u zajištění integrity dat a má i stejnou funkci (zajistit, že zprávu nikdo nezměnil). Pro ověření odesílatele si příjemce obstará od odesílatele **veřejný klíč**, dešifruje zašifrovanou charakteristiku, vygeneruje charakteristiku příchozí zprávy a porovná je. Pokud jsou stejné, nedošlo po cestě ke změně zprávy, ale především odesílatel je opravdu ten, který zprávu podepsal, protože **jen on vlastní privátní klíč**, kterým byla zašifrována charakteristika.

Samozřejmě to není tak jednoduché, vygenerovat pár asymetrických klíčů si může kdokoli a poté se **podvodně** vydávat za někoho, kým není. Aby k tomu nedocházelo, musí mít odesílatel k veřejnému klíči **certifikát**, který ověřuje jeho identitu. Certifikáty vydávají **certifikační autority**, které při tomto procesu ověří žadatelovu identitu a **podepíší jeho veřejný klíč** svým soukromým. Opět by se dalo namítat, že potenciální útočník si může vytvořit i vlastní certifikační autoritu a **certifikovat si falešné digitální podpisy**. Tomu se dá předejít jen tak, že příjemce si ověří nejen identitu vlastníka veřejného klíče, ale i **pravost certifikační autority**. Ta se zajišťuje tak zvaným **řetězcem důvěry**, který funguje na principu, že

nadřazené certifikační autority podepisují klíče svým podřazeným certifikačním autoritám.

Tento řetězec končí u **kořenové certifikační autority**, která již nemůže být dále ověřena, ale vzhledem k tomu, že je pouze jedna, nedá se o její pravosti pochybovat.

## Ochrana proti přehrání (replay attack)

Přehrání je druh útoku, u kterého útočník zachytí jinak **validní zabezpečenou komunikaci** a snaží se ji použít **opakovaně** nebo ji pouze pozdržet. Jako ochrana proti tomuto typu útoku se nejčastěji používají **časová razítka**. Podle časového razítka se určí stáří zprávy a zprávy starší než stanovená hodnota jsou ignorovány.

K tomu, aby časová razítka fungovala, musí být zajištěno, že příjemce i odesílatel mají **synchronizovaný čas** a že maximální stáří zprávy zhruba odpovídá době jejího doručení. U synchronizace času mohou být komplikací rozdílná časová pásma, ve kterých se respondenti nacházejí. Proto se obvykle pro časová razítka používá koordinovaný světový čas. Další problém synchronizace času může být v tom, že každý respondent používá jiný zdroj hodin, který je nepřesný a k tomu nepřesný s opačnou fází, nebo že útočník zaútočí přímo při synchronizaci času a podvrhne nesprávný čas. U stanovení maximálního stáří zprávy je problematická proměnlivá doba jejího doručení, která je způsobena především změnami v zatížení internetové sítě. Z toho je patrné, že pokud bude útočník dostatečně rychlý, může se mu povést i přes použití časového razítka zprávu replikovat. Proto může být časové razítko ještě doplněno například o **sekvenční číslo** zprávy nebo o nějakou jednorázovou informaci. Jak sekvenční číslo tak jednorázová informace vyžadují **uchování stavu**, což může být problematické, protože komunikace na internetu bývá často nestavová.

# Unicast, Broadcast, Anycast a Multicast

(Pro jistotu, protože to nikde jinde nezaznělo.)

## Unicast

Jedná se o komunikaci jednoho odesílatele s jedním příjemcem (**one-to-one**). Komunikace je řešena standardním směrováním a jedná se o **nejpoužívanější** druh komunikace. Typická unicast komunikace je např. **HTTP**, **SMTP**, ale i **Video on Demand** (YouTube).

## Broadcast

Jde o komunikaci od jednoho bodu ke všem ostatním bodům v síti (**one-to-all**). Broadcastové IPv4 adresy mají všechny **bity adresy hosta** (host address) **rovny 1**. Pozor **IPv6 nemá broadcastové adresy** (vše je **řešeno přes multicast**). Na linkové vrstvě je MAC adresa pro broadcast **ff:ff:ff:ff:ff:ff**. Broadcast se používá např. u **ARP** (Address Resolution Protocol – získání MAC adresy pro danou IP adresu) nebo **DHCP** (Dynamic Host Configuration Protocol – dynamické nastavení parametrů síťového zařízení: IP adresa, DNS server, defaultní brána, ...). Lze zasílat pouze přes **UDP**.

## Multicast

Multicast je druh komunikace od jednoho vysílajícího k více příjemcům, kteří se přihlásili k dané multicastové skupině (**one-to-many**). Odesílatel ale zasílá pouze **jeden datagram**, jeho **duplikaci a šíření řeší síťové prvky**. Přihlašování do skupin je řešeno:

- **IPv4** pomocí protokolu **IGMP** (Internet Group Management Protocol) a zpráv **Membership Report** (přihlášení se) a **Leave Group** (odhlášení se),
- **IPv6** pomocí protokolu **MLD** (Multicast Listener Discovery) a zpráv **Multicast Listener Query** (test, jestli někdo ještě naslouchá), **Multicast Listener Report** (přihlášení do skupiny) a **Multicast Listener Done** (odhlášení).

Existují vyhrazené IP adresy (L3) pro multicast:

- **IPv4** blok adres **D**, tj. **224.0.0.0** až **239.255.255.255**,
- **IPv6** adresy s prefixem **FF00::/8**.

Multicast na L2 je řešen **mapováním IP adres** na **MAC** adresy, mapování ale není přesné:

- **IPv4** používá pro mapování 23 bitů MAC adresy, 5 bitů se nemapuje ( $32 - 23 = 9$  (blok D) = 5, tj.  $2^5 = 32$  multicast IP adres je mapováno na stejnou MAC adresu), jedná se o adresy **01:00:5E:00:00:00** až **01:00:5E:7F:FF:FF**.
- **IPv6** používá pro mapování 32 bitů MAC adresy a jedná se o adresy **33:33:xx:xx:xx:xx**, tedy **33:33:00:00:00:00** až **33:33:FF:FF:FF:FF**. [[media/szz-44/media/image4.png]]

Multicast se používá pro **streamování televize** a **rádía** (IPTV, IP Radio) a zejména u směrovacích protokolů **RIP**, **EIGRP**, **OSPF**, **DVRMP**, ... U IPv6 nahrazuje multicast broadcast, např. při konfiguraci IP adresy pomocí **NDP** (Neighbour Discovery Protocol – obdoba DHCP pro IPv6, když nepoužijeme DHCPv6). [[media/szz-44/media/image7.png]]

## Anycast

Na síti existuje **více serverů**, které poskytují **stejnou odpověď** (CDN). Typicky je anycast provoz směrován k nejbližšímu serveru, ale v případě poruchy může být použit jiný (nebo klient si případně může vybrat jaký). Anycast slouží také jako ochrana vůči DoS útokům a umožňuje **rozložení zátěže**.

## Zdroje

- SZZ okruh 44 — studijní materiály FIT BUT (szz-44.docx). Obrázky: media/szz-44/.

---

**Navigace:** [\[\[index | • Přehled okruhů\]\]](#) · [\[\[okruhy/43-tcp-ip-komunikace | 43. TCP/IP komunikace\]\]](#) · [konec přehledu](#)